



EXPOUND

EXPOUND LABORATORIES™

Finality Assurance™

An Architecture for Computing Reliance

Kenneth Lawrence Cheney

Expound Laboratories™ · expound.com

Version 1.0 · August 26, 2026

© 2026 Kenneth Lawrence Cheney. All rights reserved.

1. The computer said APPROVED

A screen shows one word.

APPROVED

You know what it means. Everyone knows what it means. That is the problem.

Consider an invoice for \$40,000. The record says APPROVED. Ask what that establishes, and the single word turns out to be standing in for a list of separate propositions:

- An invoice was submitted.
- Its identity was established, and it is the invoice you think it is.
- Someone reviewed it.
- That person held authority.
- The authority covered this transaction, at this amount, for this purpose.
- The approval action actually occurred.
- The intended effect occurred, and the money is where it should be.
- The evidence behind all of that is still current.
- The approval still applies under the policy in force today.
- You, specifically, may now act on it.

Ten propositions. One word. And the word is the same word whether all ten hold or only the first.

Notice what has gone wrong. It is not that the record is false. The record is perfectly accurate about the thing it actually reports, which is that somebody or some workflow put that value in that field. It is that a single value is being asked to carry propositions that are not interchangeable, and a reader downstream cannot recover from that value alone which ones it carried.

This is not a complaint about enterprise software. Any system that compresses a decision into a label inherits the same shape, and most systems do, because for most of computing history the compression was affordable. A person could reconstruct the missing propositions when it mattered, because results arrived at a pace a person could keep up with.

The claim of this article is narrower and more useful:

Producing an answer and establishing that someone may rely on it are different computational problems.

Computers have become extraordinarily good at the first. They calculate, search, rank, predict, generate, recommend, and increasingly act. The second problem has been left to people, to process, and to reconstruction after the fact.

Finality Assurance™ is an architecture for computing the second one.

2. Reliance is already a serious problem

The question of when to rely on a machine has a substantial research tradition.

The canonical review is Lee and See's *Trust in Automation: Designing for Appropriate Reliance*, which examines why people fail to rely on automation appropriately. Their observation is that trust guides reliance precisely where it is hardest to check: "trust guides reliance when complexity and unanticipated situations make a complete understanding of the automation impractical." They also treat appropriateness as contextual, considering "how the context, automation characteristics, and cognitive processes affect the appropriateness of trust."

That is the right starting point, and it locates the problem honestly. Humans have always had to decide when to rely on automation, and a mature literature studies how well they do it.

The architectural question is adjacent to that one rather than a replacement for it:

The question is no longer only whether people trust machines appropriately. Increasingly, machines have to compute what reliance is warranted, for the exact question at hand.

3. From trust to computed reliance

Four words get used interchangeably in practice and mean different things. Separating them, and then adding a fifth, is the first piece of vocabulary this article asks you to take away.

Trust is a disposition to accept under uncertainty. It is held by a person or an organization toward a system, and it is not a property of any particular result.

Confidence characterizes assessed uncertainty, expected correctness, evidence strength, or maturity: a probability, an uncertainty bound, a prediction set, an evidence grade. It may describe an individual result or a broader population.

Authorization is a decision about permission. It answers whether an actor may do a thing.

Reliance is the use of a claim or result as the basis for consequential action. It is about one result, one actor, one purpose, at one moment.

Computed reliance is a machine-evaluable determination of whether the governed basis for that reliance is satisfied, for the exact question at hand.

Finality is the condition of a claim whose governed basis for a named reliance has been established. It is held rather than reached, which is why P11 recomputes it rather than remembering it.

The distinction that does the most work is between confidence and reliance. What confidence does not do by itself is grant authority to rely. A model being correct ninety-nine times in a hundred does not, by that statistic alone, answer whether this specific result is authorized for this specific consequential use. Nor does a well-calibrated number attached to this very result. Calibration answers how often claims like this one are correct. Reliance asks whether this party may act on this one, for this purpose, with these consequences.

This is also the point at which it is worth saying what reliance actually depends on:

Reliance asks whether a particular claim may support a particular action, by a particular relying party, for a particular purpose, under the relevant authority, evidence, conditions, time, and consequences.

Adjacent work treats trustworthiness as resisting compression into a single property. The NIST AI Risk Management Framework treats it as multidimensional and contextual, listing characteristics of trustworthy AI systems as "valid and reliable, safe, secure and resilient, accountable and transparent, explainable and interpretable, privacy-enhanced, and fair with harmful bias managed," and noting that "creating trustworthy AI requires balancing each of these characteristics based on the AI system's context of use."

Those are examples of properties that may matter to reliance. Finality Assurance is concerned with something narrower and more mechanical: how the evidence, authority, meaning, time, scope, and consequences relevant to an exact reliance question are brought together without collapsing distinct decisions into one label.

4. Why first principles, and what a ground is

Most architectural writing presents its rules as a flat list. That is convenient and it hides something important, because the rules in any real architecture do not all have the same standing. Some are foundational. Some follow from the foundations. Some are choices the architect made, defensibly, among alternatives that would also have worked.

Finality Assurance separates them, and publishes which is which.

A **first principle** is a proposition treated as irreducible for the reliance problem. Each is treated as irreducible for the reliance problem: the question it answers cannot be stated without it. There are twelve.

A **further governing principle** is a constitutionally binding proposition downstream of the first principles, stated separately with its **ground** published. There are also twelve. Each one names what it rests on and what kind of support that is.

The ground carries one of four statuses, and they are not interchangeable:

- **Entailed** by earlier principles.
- **A declared axiom**, adopted for stated reasons rather than proven.
- **A design election**, one justified choice among several.
- **An implementation construction**, a mechanism elected to discharge an obligation.

The architecture states this as four inequalities, and they are worth carrying away:

```
FIRST_PRINCIPLE    != DERIVED_PRINCIPLE
DERIVED_PRINCIPLE  != OPTIONAL
DECLARED_AXIOM     != PROVED_RESULT
DESIGN_ELECTION    != LOGICAL_NECESSITY
```

Derived does not mean optional. A principle that follows from another still binds. And a rule held by choice is not presented as a rule held by proof, which is why the ground appears in every row below rather than in a footnote.

The reason to publish grounds is practical. A set of statements presented as equally axiomatic offers nowhere to disagree in particular, only wholesale. A set whose grounds are published can be engaged at a named joint: a reader who accepts the first twelve and disagrees with a consequence has a specific place to stand. That is the difference between a constitution and a manifesto.

5. The twelve first principles

Each is irreducible for the reliance problem. Identifiers are stable across the Finality Assurance family, which is why the numbering below is not sequential.

P1. Reality is independent of its representation

The record is not the thing.

A database row can represent a payment. It cannot make the payment happen by being written. A dashboard can report an approval; the reporting does not create the approval. A status, a report, a projection, or a confidence statement about a condition does not bring that condition about, and none of them creates the finality decision.

There is one exception and it is precise. An **authorized performative act**, such as an executed contract, an enacted rule, or a signed directive, may create an institutional fact. Signing the contract does make the agreement exist. Where that happens, the act is typed and evaluated as an issuance event rather than mistaken for evidence that some other fact already held. The distinction matters because the two look alike on a screen and behave completely differently: an issuance creates the fact, a report describes one.

For reliance: every representation must be traced to whatever actually established the fact, or identified as an issuance that established it directly.

P3. Meaning and identity precede evaluation

Before asking whether something is valid, fix exactly what "something" refers to.

Nine things are fixed for the evaluation at hand: subject, candidate, evidence, implementation, policy, authority, environment, scope, and decision question. Fixed means closed before the evaluation begins, or versioned so that a change opens a *new* evaluation rather than quietly altering the one in flight.

This is more demanding than it sounds. "Is the invoice approved" is not yet a question. Which invoice, which candidate claim about it, produced by which implementation, under which version of the approval policy, against which body of evidence, by which authority, in which environment, within which scope, for which decision? Until those nine are pinned, there is nothing to evaluate, only something to argue about.

For reliance: an answer inherits the identity of the question. Change any term and you have a different answer, not an updated one.

P4. Authority, capability, and admissibility are different

Can is not may. And may in general is not may in this instance.

A credential or a technical ability does not by itself establish that the exact transition is permitted now for the stated purpose. Three separate facts hide inside most permission checks: whether an actor is able to act, whether the actor is permitted to act, and whether the act is allowable in this instance.

Being able to issue the payment instruction is a fact about system access. Holding approval authority is a fact about the actor's role. Whether *this* payment, *now*, *for this purpose* is allowable is a third fact, and none of the three implies another on its own.

Where a policy binds them together, that is legitimate, and it is worth being exact about what carries the implication: **the binding carries it, not the capability**. A policy may say that holders of this role may approve up to this amount. Then the capability implies admissibility because the policy said so, and the policy is the thing to inspect.

For reliance: the reliance question is always indexed to an exact transition, a moment, and a purpose.

P5. Runtime decision dimensions remain separate

Admitted, executed, occurred, and reliable are four different findings, not four words for one.

Reliable, not reliable. Reliable describes a system's general tendency. Reliable is a property of one result, for one party, for one purpose, right now, and the distinction is the whole subject.

Eight decisions stay distinct: **admission, execution, occurrence, outcome, canonicity, assertion, reliance, applicability**.

The important property is not that they form a sequence. It is that they do not.

Each decision can succeed while its neighbors fail. They are not stages of a single pipeline.

A transition may be admitted and never occur. It may occur and produce the wrong outcome. It may produce the right outcome and be described inaccurately. It may be described accurately and still be unusable by a particular party for a particular purpose. It may be usable at eleven o'clock and unusable at noon.

Two of the eight are the ones most often missing from a design. **Canonicity** asks which representation is authoritative when several exist. **Applicability** asks whether the verdict still holds now.

For reliance: keeping these decisions separate preserves information the exact reliance question may later require. Once several of them have been compressed into one undifferentiated label, a downstream consumer cannot determine from that label alone which finding was actually made.

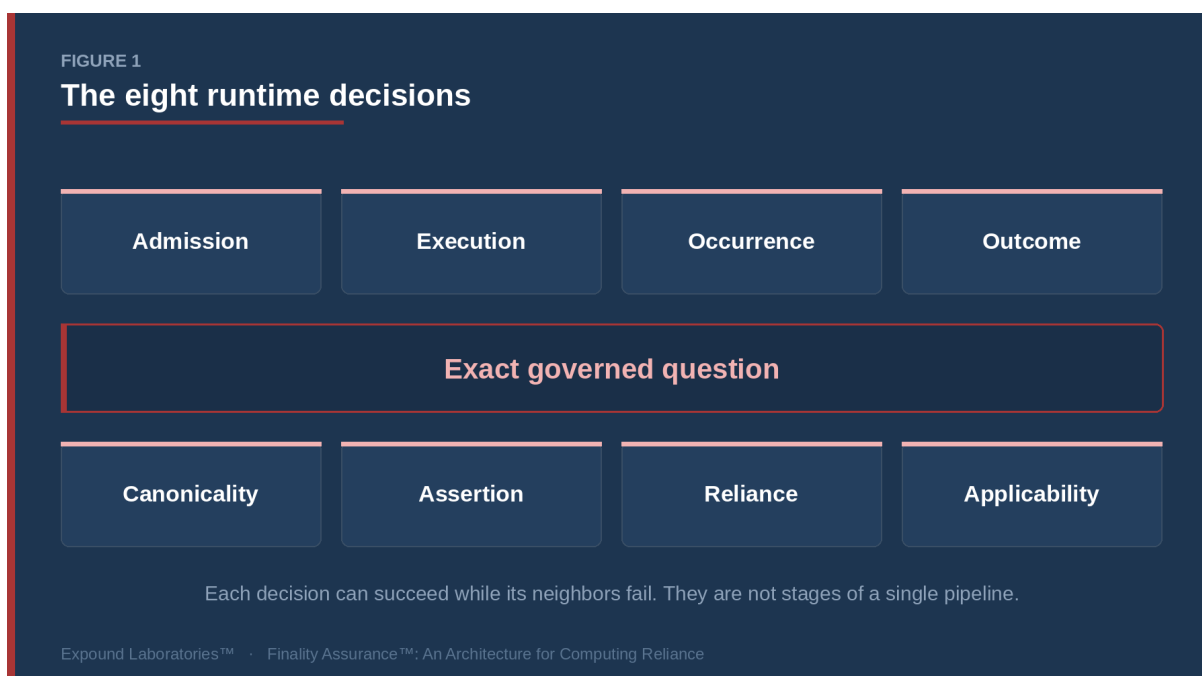


Figure 1 The eight runtime decisions

P8. False, unknown, conflicted, stale, unavailable, and not-applicable are distinct

Unknown is not false. Stale is not current. Conflict is not certainty.

Absence of proof is not proof of falsity. No evidence of failure is not evidence of no failure. Six dispositions, and each carries a typed consequence, which is to say a different instruction about what to do next.

"We could not reach the sanctions screening service" and "the sanctions screening returned clean" are opposite findings, and a Boolean field records them identically. So does a system that treats "not applicable to this transaction type" as a pass. The instruction each disposition carries is different in kind. False says the thing did not hold. Unknown says go and find out. Unavailable says the path itself is broken and the finding is about your infrastructure rather than the transaction. Conflicted says two sources disagree and somebody has to resolve which governs. Stale says this was true and its warrant has lapsed. Not-applicable says stop asking, correctly. Collapse any two and the next system in line receives an instruction that was never issued.

For reliance: merging any two of the six destroys a distinction no later step can recover.

P9. Completeness is relative to an anchored evidence and source universe

"We checked everything" means nothing until "everything" is defined.

A completeness claim is valid only relative to a named universe of evidence and sources, fixed and attributable before the check runs. A background check can be complete against every database it was required to search and say nothing whatever about a database that was never in its required universe. Both statements are true at once, and only one of them is usually reported.

The clause that does the real work is the constraint on who gets to set the universe:

Callers cannot shrink the required universe.

If the party asking the question can narrow what counts as everything, then "all required checks passed" degrades into "all the checks I asked for passed," and a system that requires nothing passes everything.

For reliance: completeness travels with the universe it was computed over, or it does not travel.

P11. Historical validity is not current applicability

True then and reliable now are two questions, and answering one does not answer the other.

Two propositions about two different times. The record of what was known and decided then remains what it was, byte for byte. Present reliance is recomputed against current evidence, authority, identity, policy, and time.

An approval granted in March under March's policy was validly granted. That is a permanent historical fact and nothing should be allowed to alter it. Whether anyone may rely on it in August is a different question with a different answer, and asking the first question does not answer the second.

The architecture calls the maintenance of that second answer Continuous Finality™.

For reliance: history is preserved and authority is recomputed. Neither substitutes for the other.

P13. The subject cannot be its own sole judge

Self-emitted evidence is admissible. It cannot alone establish the subject's correctness or independence.

A producer, executor, model, or adjudicator **may emit evidence**. That evidence is real and useful, and nothing here says to discard it. What such a party cannot do is *alone* establish its own correctness, or its own independence.

The second half is the one that gets missed. A component that reports it is independent has not established that it is independent; it has reported it. Independence is a property of the relationship between parties, and a party is not positioned to certify its own.

For reliance: self-emitted evidence is one input among several, never the sole basis, and never the establishment of the independence that would make it sufficient.

P15. Reliance is query-relative, consequence-aware, and scoped

Reliable for whom, to do what, for what purpose, with what consequence?

A finality decision answers one observer-action-purpose-consequence question, not every possible question. Four coordinates, and purpose is separate from action: displaying a figure on an internal dashboard and releasing a payment are different actions, and publishing a draft internally and filing it with a regulator are different purposes for the same action.

There is no universal verdict that licenses everyone to do everything. An answer sufficient to schedule a review may be insufficient to move money, and the same evidence supports both statements without contradiction. Consequence is the coordinate most often left implicit, and it is the one that changes what counts as enough. The same invoice, the same evidence, the same authority: a reversible internal booking and an irreversible outbound payment are different reliance questions, and an architecture that cannot tell them apart is answering whichever one it happened to be configured for.

For reliance: the question is part of the answer. A verdict lifted away from the question it answered is not a weaker verdict, it is not a verdict.

P17. Control presence is not control capability

A control counts for what it establishes, not for what it is called.

A checker, rollback, receipt, reviewer, or signature is load-bearing only after three things are established: **discrimination** (it can tell the difference between the cases it claims to distinguish), **nondegeneracy** (it does not pass everything, or fail everything), and **current applicability** (it applies to this case now).

Where a control claims prevention, enforcement, rollback, or compensation, there is one more bar: the claimed intervention must be shown to occur. A control that claims to reverse something and has never been observed reversing anything is claiming behavior nobody has seen.

The carve-out is important and easy to lose. **A passive observer or a read-only checker claims no intervention and is not held to one.** A monitor that reports is doing exactly what it says.

For reliance: the inventory of controls is not the finding. What each control discriminates is, and a control list that has never been tested against that question is a description of a budget.

P19. Local finality does not compose automatically

Correct parts do not automatically make a correct whole.

Component verdicts compose only under explicit contracts of six kinds: **assumption-guarantee, interface, temporal, authority, dependency, and common-cause.**

Service A interprets a timestamp as UTC. Service B interprets the same field as local time. Service C evaluates against a policy version neither of the others has loaded. Each is correct under its own assumptions. The system-level conclusion depends entirely on whether those assumptions compose, and nothing in the components' own correctness answers that.

The class readers most often omit is the last. **Common-cause** covers correlated failure: three independent checks that all read the same stale cache are not three checks.

For reliance: composition is available, and it is an argument that must be supplied rather than a property that is inherited.

P20. Timing, load, failure, false refusal, and liveness are part of correctness

A system that answers correctly but too late has not answered.

Safety without usable resolution, timely invalidation, bounded stale reliance, or safe degradation is operationally incomplete. The cleanest way to see why is the limiting case:

A system that refuses everything may satisfy a narrow prohibited-transition safety property and still be operationally incomplete, because it has no liveness and no utility.

Nothing bad gets through. Nothing at all gets through. If refusing everything can satisfy the definition of correct, the definition is wrong, which is why liveness, timeliness, and false refusal sit inside the definition of correct rather than beside it. False refusal is a real cost, and an architecture that treats it as free has moved the cost rather than removed it.

For reliance: an answer that cannot be obtained, maintained, invalidated, and refreshed within the conditions that make it useful is not yet an answer.

6. The twelve further governing principles

Each is constitutionally binding and each names its ground. The ground is stated once, in the row where it belongs, because a reader who accepts the first twelve and disagrees with a consequence should be able to find the joint.

P2. One consequential concept resolves to one canonical semantic authority for the evaluation at hand

One decision needs one controlling meaning.

Ground: P1 and P3, entailed. If meaning must precede evaluation and the record may be wrong, two unresolved authorities for one concept make the evaluation undefined rather than merely inconsistent. There is no fact of the matter about which "approved" was meant.

The scope clause is doing real work. **For the evaluation at hand.** This does not require one physical source of truth, and organizations that read it that way have read it too strongly. Multiple upstream sources are admissible exactly when a governed resolution yields one authoritative interpretation for the exact query and epoch. Three systems may hold customer records. What may not happen is that a reliance decision proceeds without resolving which reading of "customer" governs this question, today.

P6. The transition is the universal governance object

Govern the point where something changes.

Ground: P5 and P15, design election argued from fit. If the runtime dimensions stay separate and reliance is query-relative, the transition is where they meet. Electing it as the governed unit is a design commitment argued from that fit, binding by declaration rather than entailed.

The alternative candidates are real: govern the object, govern the actor, govern the session, govern the record. The transition is chosen because it brings the relevant coordinates together at the point where governed state changes. Something changes, at a moment, by an actor, under an authority, with consequences, and that is the set of coordinates a reliance query is indexed on.

P7. Evidence is multidimensional (P7a) and noncompensatory (P7b)

Strength on one requirement is not evidence about another.

Ground: P7a follows from P8. P7b is a declared axiom.

Multidimensionality is entailed: if the dispositions are distinct, the evidence that resolves them cannot be a single scalar. Six distinguishable findings do not compress into one number without losing exactly what distinguished them. Excellent evidence of identity does not establish authority. Excellent evidence of authorization does not establish occurrence.

Non-compensation is declared. Non-compensation is adopted as a declared axiom of this architecture rather than derived. It is the rule that keeps a strong showing on nine requirements from carrying a tenth that was never met.

P10. Domain-model adequacy is continuously governed

The model of what matters has to stay current too.

Ground: P9 and P11, entailed, with continuous governance the elected mechanism. An anchored universe that can become inadequate over time must be re-established rather than assumed.

Note the strength of the verb. Not "may evolve" but must be re-established. A required-evidence universe fixed three years ago and never revisited is not neutral; it is a claim about today's hazards made with three-year-old information, and nothing about it announces that it has gone stale.

P12. Corrections preserve history, conflict, and causal lineage

A correction adds to the record. It does not replace it.

Ground: P1 and P11, entailed. If the record may be wrong and time matters, a correction that erases its predecessor destroys the evidence that the correction was needed.

The precise requirement is worth stating exactly, because it is more permissive than it first sounds. What is derived is that **the lineage must survive in a governed record reachable from the correction**, not that the correction object itself must carry it. The corrected value does not have to drag its history around. Someone holding the correction has to be able to get to that history.

P14. Narrative, claims, and projections arising from the reporting path cannot originate authority

Reporting a decision is not making one.

Ground: P13, entailed. A description of a decision is not the decision. Admitting it as one reintroduces self-attestation through the reporting path.

This returns to the screen that said APPROVED. The dashboard displays authority; it does not manufacture authority by displaying it. And the qualifier matters: the principle governs claims

arising from the reporting path. An authorized issuance event, a statute, a contract, a signed directive, is an authority act rather than a report, and is untouched by this rule. The test is not whether something appears on a screen. It is whether the act is constitutive or descriptive.

P16. Reliance creates a transitive consequence graph

Reliance creates dependencies. It does not create permissions.

Ground: P15, entailed. If reliance relates a party, a purpose, and a claim, then relying on what relied on something else is an edge, and edges compose.

What composes along those edges is precise: **consequence** does. Invalidation and notification reach through them. If A is withdrawn, the parties who relied on B, which relied on A, are reachable and can be told.

Permission does not transfer along them. That C relied on B, and B was permitted to rely on A, establishes nothing about whether C may rely on A directly. This is the point where reliance graphs are most often built wrong, because consequence propagation and permission inheritance look like the same edge and are not.

P18. Confidence and optimization are advisory, never dispositive (P18a derived, P18b declared)

Confidence informs a decision. It does not create authority.

Ground: P18a follows from P4 and P13. P18b is a declared axiom.

P18a is entailed and the derivation is short. A score produced by the system being evaluated is capability speaking about admissibility, which P4 separates and P13 forbids.

P18b covers the harder case: a score produced **outside** the evaluated system. Nothing in P4 or P13 rules that score out, and this architecture still admits none as dispositive, so admissibility always resolves to typed evidence.

None of this is a criticism of probabilistic systems. A calibrated confidence, a prediction set, a ranking, a reward, an abstention decision: each is real work and each can inform an evaluation. None of them is a permission, and the architecture declines to let any of them become one.

P21. Production claims require exact implementation refinement and outside-in rederivation

Assurance follows the exact thing being relied upon, and is re-established from outside it.

Ground: P13 and P17, entailed, with outside-in rederivation the elected construction. If the subject cannot judge itself and presence is not capability, a production claim must be re-established from outside the thing claiming it.

Two halves, and the second is the one most often dropped. **Exact implementation refinement** is the first: assurance about a design, a specification, a model, or an earlier artifact does not silently migrate to a different object, and the claim stays bound to the exact subject it was established about. **Outside-in rederivation** is the second: the re-establishment is performed from outside the thing making the claim, and it is the construction this architecture elects to discharge that obligation.

P22. Product surfaces must be non-bypassable for the transitions they govern

One ungoverned path breaks the composition argument for every governed one.

Ground: P17 and P19, entailed. A control that can be routed around is present without being capable, which is exactly P17. And the composition argument is unforgiving: **one ungoverned path breaks the composition argument for every governed one.**

It is not that a bypass weakens governance proportionally. It is that a single unmediated route removes the basis for the system-level claim, because the claim was that the governed paths were the paths.

P23. Semantic versions and migrations are explicit

Meaning has versions.

Ground: P3 and P11, entailed. If meaning precedes evaluation and meaning can change over time, then the version of the meaning is **part of the identity of the claim**, not metadata attached to it.

If "approved" is redefined in June, then a March approval and a July approval are claims about different things wearing the same word. Migrations are explicit for the same reason: a claim evaluated under one version does not silently become a claim under the next.

P24. The canonical output is a typed Finality Account; the Finality Verdict is its action-specific categorical projection

Rich inside. One answer out. Nobody types it.

Ground: P5 and P8, entailed, with the typed packaging a design election. If the dimensions stay separate and the dispositions are distinct, the retained object must be multidimensional, and any single categorical answer is a projection of it. That much is derived. That the object is packaged as a typed Account with a categorical projection is the election this architecture makes of it.

Internally the Finality Account keeps the eight runtime decisions of P5 separate, together with the evidence, authority and currentness findings behind them, because P5 and P8 both hold that merging two distinct findings destroys something no later step recovers.

Externally, the exact question receives one categorical answer. That answer is a **projection**, and it is **not a writable status flag**. Which is where the article began: a word on a screen that somebody put there. The Finality Verdict is the same shape and the opposite thing. It is computed from the Account for one question, and nobody types it.

7. From world to reliance

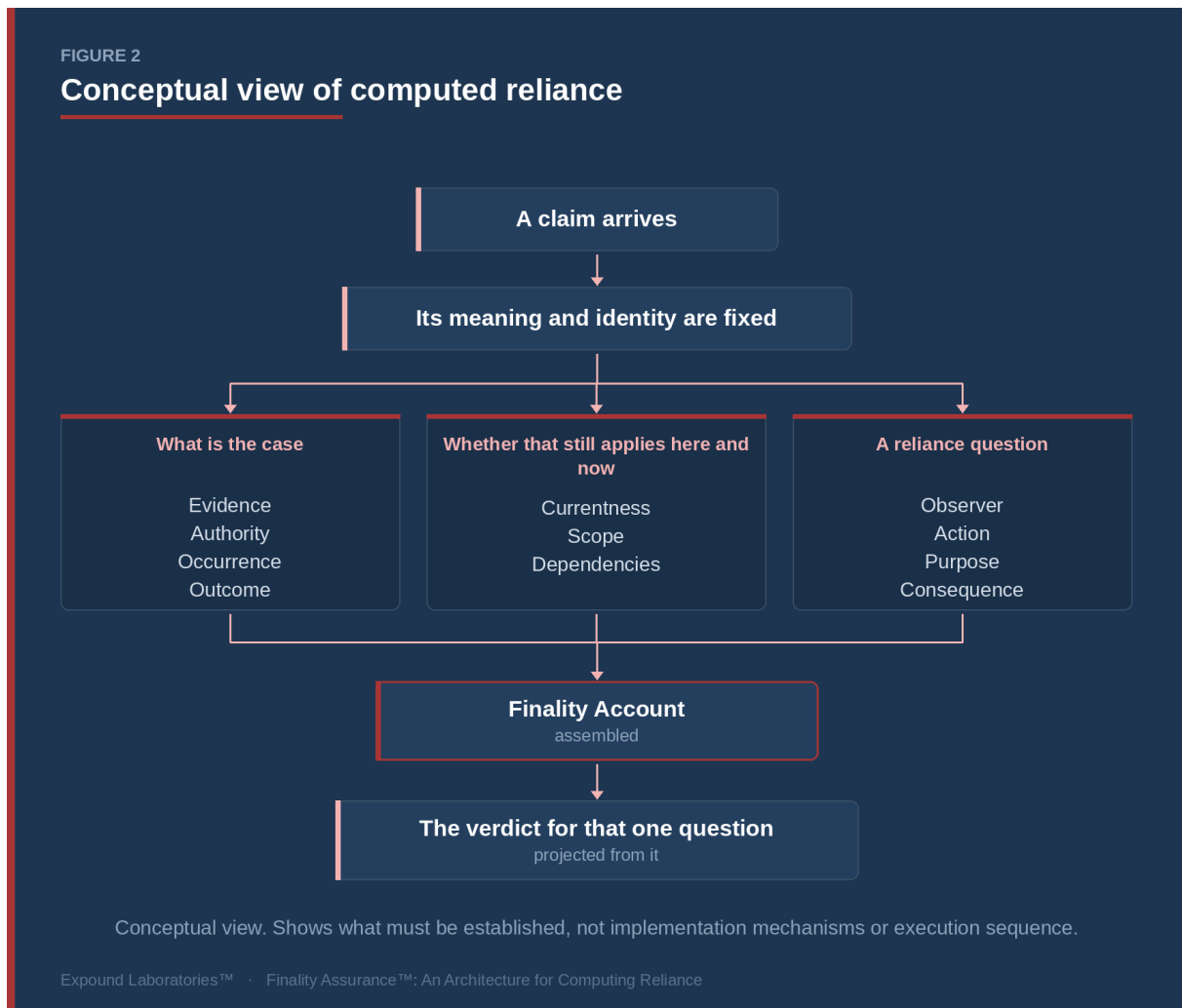


Figure 2 Conceptual view of computed reliance

The shape is worth stating in words as well. A claim arrives. Its meaning and identity are fixed. Evidence, authority, occurrence and outcome establish what is the case. Currentness, scope and dependencies establish whether that still applies here and now. A reliance question names its observer, action, purpose and consequence. From all of that a Finality Account is assembled, and the verdict for that one question is projected from it.

This article presents the conceptual model. It says what has to be established, not how any particular system establishes it.

8. Finality Assurance in the larger computing landscape

Finality Assurance sits alongside several mature areas of computer science and systems engineering that address important parts of consequential decision-making.

Authentication establishes identity. Authorization governs permission. Provenance records origins and derivations. Runtime verification evaluates executions against specified properties. Temporal systems represent change across time. Human-factors research studies appropriate reliance on automation. Complete mediation ensures authority is checked wherever protected objects are reached.

Finality Assurance brings a distinct question into focus: how to compute the governed basis for reliance on an exact claim, for an exact consequential use.

Trust and automation. Lee and See show that appropriate reliance depends on trust, system characteristics, cognitive processes, and context, and that trust guides reliance most where complete understanding is impractical.

Attribute-based access control. NIST describes authorization as determined "by evaluating attributes associated with the subject, object, requested operations, and, in some cases, environment conditions against policy, rules, or relationships." The decision object is an entitlement to act, evaluated before the act.

Models extending access control through the duration of use. Park and Sandhu developed models that extend access control with authorizations, obligations, conditions, decisions that continue during use, and attributes whose values change as a consequence of access.

Zero trust. NIST's model assumes no implicit trust granted to assets or accounts based solely on network location or asset ownership, and treats authentication and authorization as "discrete functions performed before a session to an enterprise resource is established."

Provenance. W3C PROV represents "information about entities, activities, and people involved in producing a piece of data or thing, which can be used to form assessments about its quality, reliability or trustworthiness." Provenance describes how something came to be.

Runtime verification. Havelund and Roşu describe it as "the discipline of analyzing program executions using rigorous methods," covering monitoring against formal specifications and the computation of information from execution traces.

Event sourcing. Fowler's pattern captures "all changes to an application state as a sequence of events," allowing state to be rebuilt by replay and past state to be determined at any point in time.

Information protection. Saltzer and Schroeder's complete mediation principle: "Every access to every object must be checked for authority."

Trustworthy AI. The NIST AI Risk Management Framework treats trustworthiness as multidimensional and contextual rather than reducible to one property.

These fields illuminate different dimensions of consequential computing: identity, permission, provenance, execution, time, trust, and mediation. Finality Assurance addresses a distinct architectural question: whether the governed basis for reliance on an exact claim, by an exact party, for an exact consequential use, is satisfied now.

9. Why computing reliance matters more now

The reliance problem is older than AI and does not depend on it. Approvals were being over-read before any of this.

What has changed is proportion. For as long as consequential results were produced at human pace, they could be checked at human pace, and the gap between "the system says this is finished" and "someone established that it is" was absorbed by people doing the establishing by hand. Machine production breaks that proportion. Systems now recommend, classify, predict, draft, approve, route, execute, adapt, and coordinate, and the volume of things that might be relied upon stops tracking the number of people available to establish whether they should be.

The AI era did not invent the reliance problem. It industrialized the production of things people may be tempted, expected, or required to rely upon.

Which leads to the claim this article exists to make:

As machine-produced outputs become abundant, the scarce resource increasingly becomes justified reliance.

There is a second reason, more specific to autonomous operation. When software acts on its own, the interval in which a person might have decided does not exist unless the system creates one. Approval gates and pre-execution checks create exactly that interval. What they need is something to compute in it.

10. The scope of computed reliance

Finality Assurance computes reliance over a defined question, evidence basis, authority structure, semantic scope, temporal context, and consequence. Its object is bounded by design rather than universal by assertion.

Positively stated, that means: exact questions have exact scope. Evidence has defined provenance and applicability. Authority is explicit. Currentness is evaluated rather than assumed. Dependencies are represented. Reliance is tied to an observer, an action, a purpose, and a consequence. The resulting verdict answers that governed question.

Soundness is relative to a declared model: the stated assumptions, the source agreements, the observation contracts, the semantic mappings, and the deployment boundaries. A declared model can be inspected and versioned; an undeclared one cannot.

Finality Assurance makes the basis of its answer explicit and machine-evaluable for the governed reliance question.

11. Where computing reliance matters

The applicability test is not an industry. It is whether a decision has conditions and evidence that can be identified for the question at hand. That is decided decision by decision, and the list is longer than most organizations have counted. It rarely follows the boundaries of a department.

Financial operations, before money moves. **Software delivery**, before code or configuration becomes authoritative. **AI-assisted decisions**, before machine-produced work becomes the basis for consequential action. **Healthcare and safety processes**, where evidence, authority, timing and scope materially affect what is permissible. **Legal and compliance workflows**, where authority and evidence must stay tied to particular purposes and actors. **Autonomous and semi-autonomous systems**, where machines participate in state transitions people previously reviewed by hand. **Enterprise workflows**, wherever "approved," "complete," or "done" currently requires expensive human reconstruction to interpret.

Bounded decisions of that kind appear inside activities that are not themselves bounded: whether a result may be published, whether a data set may be used, whether a finding may support a further conclusion, whether an artifact may enter a governed workflow.

12. How the principles are reasoned

The set is published with its grounds because that is what makes it a constitution rather than a manifesto.

Four statuses appear, and they are not interchangeable: entailed by earlier principles; a declared axiom adopted for stated reasons; a design election among justified alternatives; an implementation construction elected to discharge an obligation. A row's authority is exactly its stated ground.

Publishing the ground preserves the reasoning structure of the architecture. A reader can distinguish what Finality Assurance treats as foundational, what follows from those foundations, what is adopted as an axiom, and what is elected as an architectural construction. That makes the relationship among the twenty-four propositions explicit rather than flattening them into a single undifferentiated list.

The principles are meant to be understood together with the grounds that support them.

13. From computing answers to computing reliance

Return to the screen.

APPROVED

The word has not changed. What has changed is that the questions behind it are now separable, and each one has a name.

What does approved mean here, and under which version of that meaning? Which exact subject? Which authority, and did it cover this transition, now, for this purpose? Did the thing occur, and was the outcome the intended one? Which representation is authoritative? What evidence establishes each of those, drawn from which universe, and who was permitted to define that universe? Is any of it still current? Which dependencies could invalidate it, and who would be told? Who intends to rely on it, for what action, for what purpose, and with what consequence?

None of those questions is exotic. Every one of them is asked, somewhere, by someone, usually late and usually by hand.

Computers have spent decades becoming better at computing answers. They calculate, search, predict, recommend, generate, and increasingly act. Consequential action presents a second problem, and it has largely been left to people:

What are we justified in relying upon?

The twelve first principles of Finality Assurance define the foundations of that question. The twelve further governing principles describe the architecture that follows when those foundations are taken seriously.

Finality Assurance treats reliance not as a feeling, a label, a confidence score, or an inference that humans reconstruct around the system, but as a governed object that can itself be computed.

Sources

External sources referenced in this article, retrieved on the access date shown. Third parties, standards and publications named here are referred to descriptively and nominatively. Nothing implies endorsement, partnership, implementation, adoption or certification by any of them, and third-party names are the property of their respective owners.

1. **Lee, J. D., and See, K. A.** *Trust in Automation: Designing for Appropriate Reliance*. Human Factors 46(1), 2004, pp. 50-80. DOI 10.1518/hfes.46.1.50_30392 · accessed 2026-08-26
2. **Fowler, M.** *Event Sourcing*. martinowler.com, December 12, 2005.
<https://martinowler.com/eaDev/EventSourcing.html> · accessed 2026-08-26
3. **Hu, V. C., et al.** *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. NIST Special Publication 800-162, January 2014, including updates as of August 2, 2019.
<https://csrc.nist.gov/pubs/sp/800/162/upd2/final> · accessed 2026-08-26
4. **Rose, S., Borchert, O., Mitchell, S., and Connelly, S.** *Zero Trust Architecture*. NIST Special Publication 800-207, August 2020. DOI 10.6028/NIST.SP.800-207 · accessed 2026-08-26
5. **Park, J., and Sandhu, R.** *The UCON_ABC Usage Control Model*. ACM Transactions on Information and System Security 7(1), February 2004, pp. 128-174. DOI 10.1145/984334.984339 · accessed 2026-08-26
6. **Moreau, L., and Missier, P., eds.** *PROV-DM: The PROV Data Model*. W3C Recommendation, April 30, 2013. <https://www.w3.org/TR/prov-dm/> · accessed 2026-08-26
7. **Havelund, K., and Roşu, G.** *Runtime Verification, 17 Years Later*. Runtime Verification (RV 2018), LNCS 11237, Springer, pp. 3-17. DOI 10.1007/978-3-030-03769-7_1 · accessed 2026-08-26
8. **Saltzer, J. H., and Schroeder, M. D.** *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), September 1975, pp. 1278-1308. DOI 10.1109/PROC.1975.9939 · accessed 2026-08-26
9. **NIST.** *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, January 2023. DOI 10.6028/NIST.AI.100-1 · accessed 2026-08-26

On publication and rights. The concepts, questions, examples and diagnostic vocabulary in this article are published for discussion, testing, teaching and critique. Nothing in this publication grants a license under any patent or patent application, copyright, trademark or other intellectual-property right, and it grants no implementation, certification or commercial license of any kind. These rights are reserved. The architecture is the subject of United States patent applications and also includes separately retained trade secrets, copyrighted materials and marks. Nothing here asserts that any party infringes or that any party is free to operate. Certain implementation mechanisms remain confidential and are not disclosed here.

Availability. The Finality Assurance family of standards is available for licensing under commercial terms. Nothing in this article is an offer, a commitment, or a guarantee of terms.

Terminology. Finality Assurance™, Continuous Finality™ and Expound Laboratories™ are claimed marks of Kenneth Lawrence Cheney. They name that work and are not generic labels for a class of product.

Scope of this article. This is a conceptual explanation of an architecture, and the confidential implementation mechanisms are outside its disclosure boundary. The principle statements, their grounds and their identifiers follow the public technical paper and the core technical article. Nothing here is legal, regulatory, clinical or financial advice.

Disclosure. The author has a commercial interest in the subject of this article.